# Desafio #2.

Oscar Mauricio Echeverri Parra.

Luna Jaramillo Guzmán.

## UdeABook

## 1. Análisis del problema

Después de leer el enunciado, entendimos que debemos crear la parte lógica de una red social llamada Ude@Book. No nos están pidiendo una página web ni una aplicación con imágenes reales, sino un programa en C++ que funcione por consola y permita manejar usuarios, amistades, mensajes y álbumes de fotos. Cuando el programa necesite mostrar una foto, bastará con enseñar la ruta donde estaría guardada.

Lo que más nos llamó la atención es que casi toda la información está relacionada. Por ejemplo, cuando dos personas se vuelven amigas, la relación debe quedar registrada para ambas. También debemos tener cuidado con los mensajes, porque un mismo mensaje pertenece a la conversación del que lo envía y del que lo recibe. Por eso creemos que lo mejor es diseñar primero las clases y luego programar las opciones del menú.

Nuestra idea es dividir el problema en partes pequeñas, sin salirnos de las funciones que pide el documento. El programa trabajará con usuarios que ya están registrados y con fotografías que ya existen en archivos. Así podremos concentrarnos en la organización de los datos y en que cada operación funcione correctamente.

## 2. Qué debe permitir hacer el programa

Primero se cargarán los datos desde archivos. Luego cada persona podrá ingresar con su nombre de usuario y contraseña. Al entrar verá el menú y tres rutas de fotos de perfil tomadas de sus amigos. Podrá actualizar este grupo una sola vez. La elección debe ser aleatoria, pero con mayor posibilidad para los amigos que tengan más amigos en común.

Desde el menú se podrán enviar y aceptar solicitudes de amistad. También se podrán consultar sugerencias de amigos de amigos, ordenadas por el número de contactos en común, y enviar una solicitud a uno de ellos. La cantidad de sugerencias solicitadas será un número K entre 1 y 20.

Otra opción será visitar el perfil de un amigo y consultar sus álbumes públicos o visibles para amigos. Las fotografías de cada álbum se recorrerán como un carrusel, pasando hacia adelante o hacia atrás y pudiendo dar o quitar un like. Para los mensajes se permitirá escribir a un amigo, revisar los no leídos y buscar los recibidos entre dos fechas.

Al terminar cada funcionalidad debemos indicar las iteraciones utilizadas y la memoria que está ocupando el programa. Esta parte no es solamente un dato adicional: debemos pensarla desde el diseño para que después no sea difícil medirla.

## 3. Reglas que debemos cuidar

Antes de enviar una solicitud verificaremos que el usuario exista, que no sea el mismo que la envía, que todavía no sean amigos y que no haya una solicitud pendiente entre ellos. Al aceptar una solicitud se agregará la amistad en los dos sentidos. El documento no pide eliminar amistades ni crear cuentas nuevas, así que esas funciones no formarán parte del menú.

Los álbumes pueden ser públicos, privados o visibles para amigos. Como la visita corresponde al perfil de un amigo, se mostrarán sus álbumes públicos y los configurados para amigos, pero no los privados. Cada foto estará asociada a un solo álbum, tendrá fecha, tamaño, ruta y número de likes. Si el usuario llega a la última foto y elige siguiente, volverá a la primera; pasará lo mismo en sentido contrario.

Los mensajes solamente se podrán enviar a amigos y tendrán un máximo de 512 caracteres. No aceptaremos textos vacíos. Los mensajes recibidos que todavía no se hayan leído aparecerán ordenados por fecha; después de mostrarlos se marcarán como leídos. En una búsqueda por fechas se incluirán tanto el día inicial como el final.

También debemos considerar casos sencillos que pueden causar errores: usuarios sin amigos, álbumes sin fotografías, una lista de sugerencias vacía o personas con menos de tres amigos. Si no hay suficientes fotos distintas para actualizar las tres iniciales, lo informaremos, ya que no se pueden inventar amigos para completar el grupo.

## 4. Clases que proponemos

Para nosotros es importante que cada clase se encargue de algo concreto. No queremos poner todo dentro de la clase principal porque después sería más difícil corregir los errores y manejar la memoria. Estas son las clases que consideramos necesarias:

Plataforma: coordinará la carga de datos, el inicio de sesión y las opciones del menú. Tendrá acceso a los usuarios, mensajes y solicitudes, y buscará a los usuarios registrados.

Usuario: guardará el nombre de usuario, contraseña, correo, fecha de nacimiento, ciudad, país, fecha de inscripción y ruta de la foto de perfil. También tendrá sus amigos, sus solicitudes, sus mensajes enviados y recibidos, y sus álbumes.
SolicitudAmistad y Mensaje: la primera relacionará al que solicita con el que recibe, junto con su estado. La segunda guardará el emisor, el receptor, la fecha, el texto y si fue leído o no.
Album y Foto: Album tendrá nombre, ruta, privacidad y sus fotografías. Foto tendrá fecha de captura, tamaño, ruta y contador de likes.
FechaHora: nos ayudará a comparar fechas de mensajes y fotos sin repetir comparaciones en distintas clases.
ListaDinamica<T> y Cadena: serán clases de apoyo que construiremos nosotros mismos. La primera usará memoria dinámica para guardar elementos; la segunda manejará texto sin depender de contenedores de la STL.
Archivos y MedidorRecursos: se encargarán, respectivamente, de cargar y guardar los datos y de acumular las iteraciones y la memoria que debamos mostrar.

En el diagrama de clases representaremos estas relaciones. Plataforma tendrá los usuarios, mensajes y solicitudes; cada Usuario contendrá sus álbumes y cada Album sus fotos. En cambio, las amistades y los participantes de mensajes serán relaciones entre objetos ya existentes. Esto evitará crear una copia completa de un mismo usuario cada vez que aparezca como amigo o destinatario.
Espacio para incorporar el diagrama de clases elaborado en diagrams.net. Debe mostrar las mismas once clases, sus relaciones y sus multiplicidades.

## 5. Cómo organizaremos los datos

El enunciado prohíbe usar contenedores de la STL, estructuras struct y herencia. Por eso planteamos una plantilla ListaDinamica<T> basada en un arreglo reservado con new[]. Cuando se llene, crearemos un arreglo de mayor capacidad, pasaremos los elementos y liberaremos el anterior con delete[]. La usaremos para guardar punteros a usuarios, mensajes, solicitudes, álbumes y fotos.

Pensamos guardar una sola vez cada objeto principal. Por ejemplo, la Plataforma será responsable de liberar los usuarios, y una lista de amigos guardará referencias a ellos sin volverlos a crear. Así evitaremos gastar memoria de más y también el problema de intentar borrar dos veces un mismo objeto. Los álbumes serán propiedad de cada usuario y las fotografías serán propiedad de su álbum.

Para encontrar a un usuario por su nombre elegimos una búsqueda secuencial: revisamos la lista hasta hallar el nombre o llegar al final. No es la búsqueda más rápida si hay muchos registros, pero es sencilla de entender y de programar. Como el desafío trabaja con usuarios ya registrados, después podremos comparar cuántas iteraciones necesita esta búsqueda con otras opciones.

Por utilizar memoria dinámica debemos cuidar los constructores, los destructores y las copias de los objetos. También mantendremos los atributos privados y los métodos públicos. La clase Usuario cuenta con métodos de consulta y uno para cambiar el correo. Para las comparaciones usamos operadores sobrecargados, por ejemplo < y == en FechaHora, y también operadores de Cadena. La función amiga mismoNombre nos permite comparar dos usuarios sin hacer públicos sus datos.

## 6. Algoritmos que requieren más atención

En la selección de fotos de amigos primero contamos cuántos amigos en común tiene el usuario con cada amigo. Si son 10 o menos le damos peso 10; entre 11 y 20, peso 25; y más de 20, peso 35. Esos valores representan las prioridades 1, 2.5 y 3.5 que pide el enunciado. Después escogemos al azar tres amigos, sin repetir ninguno. Si el usuario actualiza las fotos, el nuevo grupo debe ser distinto del anterior y la actualización solo puede hacerse una vez.
Para las sugerencias recorremos los amigos de cada amigo y contamos las coincidencias. No incluimos al propio usuario, a quienes ya son sus amigos ni a quienes tienen una solicitud pendiente. Guardamos los candidatos junto con la cantidad de amigos en común y los ordenamos de mayor a menor. Finalmente mostramos hasta K sugerencias, con K entre 1 y 20, y damos la opción de enviar una solicitud.
El carrusel de álbumes será más sencillo: tendremos una posición actual y, al avanzar o retroceder, cambiaremos esa posición. Si pasa del último elemento, volverá al primero. En los mensajes compartiremos una misma referencia entre el emisor y el receptor, mantendremos el orden por fecha y cambiaremos el estado de lectura después de mostrar los nuevos.
Para las iteraciones usamos un contador en MedidorRecursos y lo actualizamos en los recorridos principales y en varios métodos auxiliares. Para la memoria sumamos los objetos y arreglos dinámicos que podemos medir desde nuestras clases. Todavía tenemos que revisar este cálculo con cuidado, porque el profesor también pide contar todas las iteraciones indirectas y la memoria de variables locales y parámetros por valor.

## 7. Archivos y organización del código

Al iniciar el programa leemos los datos desde la carpeta datos. Cuando se produce un cambio que debe conservarse, actualizamos los archivos. Las búsquedas, las sugerencias y las demás operaciones se hacen en memoria; no usamos los archivos de texto como espacio de trabajo. Separamos la información en usuarios, amistades, álbumes, fotos, mensajes y solicitudes.

El orden de los campos será el siguiente:
usuarios.txt: usuario | nacimiento | correo | fotoPerfil | clave | ciudad | pais | inscripcion
amistades.txt: usuario1 | usuario2
albumes.txt: usuario | album | ruta | privacidad
fotos.txt: usuario | album | fechaCaptura | tamano | ruta | likes
mensajes.txt: emisor | receptor | fechaEnvio | leido | texto
solicitudes.txt: emisor | receptor | estado
En solicitudes, los estados serán 0 para pendiente, 1 para aceptada y 2 para rechazada. Las rutas de las fotos seguirán la estructura /users/udebook/usuario/albums/album/foto.png, mientras que la foto de perfil estará en /users/udebook/usuario/albums/profile.png.
Para que la lectura sea sencilla, cada línea representa un registro y los campos se separan con el carácter |. Por ejemplo, una amistad se puede guardar como usuario0000|usuario0001 y un mensaje como usuario0001|usuario0000|2026-07-01|0|Hola. En los mensajes, 0 indica que todavía no fue leído y 1 indica que sí. Las fechas se admiten como AAAA-MM-DD y al guardarse también pueden incluir hora, minuto y segundo.
Para el código usaremos archivos .h y .cpp separados por clase y un main.cpp pequeño que inicie la plataforma. La plantilla de lista dinámica tendrá su implementación disponible desde el encabezado, para que el compilador pueda utilizarla con los tipos que necesitemos. No utilizaremos herencia; relacionaremos las clases mediante sus atributos y métodos.

# 8. Pruebas y conclusión

Comenzaremos probando con pocos usuarios, porque así es más fácil revisar las amistades, las solicitudes y los mensajes. Después usaremos los archivos de prueba con 1.000 usuarios. Para la demostración elegiremos una cuenta con suficientes amigos para consultar 12 sugerencias, un amigo con dos álbumes de cinco fotos cada uno y un rango de mensajes de por lo menos seis semanas. También revisaremos casos como listas vacías y personas con pocos amigos.

En conclusión, decidimos dividir el programa en clases sencillas y guardar referencias a los mismos usuarios, en lugar de crear copias para cada amistad o mensaje. Creemos que así podremos mantener el código más organizado y explicar nuestras decisiones. Nos queda comprobar con detalle el conteo de recursos y agregar el diagrama UML definitivo para que coincida con las clases y métodos del programa.